Remember, a particular policy is a map or function, so we have some sort of set of possible policies; the optimal policy is just an `argmax` (which selects the maximum) over this set of possible policies as a function of their expected rewards.

Again, the whole goal of a reinforcement learning algorithm (our agent) is to choose the actions that lead to the maximal expected rewards. But there are two ways we can train our agent to do this:

- ***Directly—*** We can teach the agent to learn what actions are best, given what state it is in.
- ***Indirectly—*** We can teach the agent to learn which states are most valuable, and then to take actions that lead to the most valuable states.

This indirect method leads us to the idea of value functions.

2.7.3. Value functions

Value functions are functions that map a state or a state-action pair to the *expected value* (the expected reward) of being in some state or taking some action in some state. You may recall from statistics that the expected reward is just the long-term average of rewards received after being in some state or taking some action. When we speak of *the* value function, we usually mean a state-value function.

Table 2.7. The state-value function

Math	English
$V_\pi: s \rightarrow E(R\|s,\pi)$,	A value function, V_π, is a function that maps a state, s, to the expected rewards, given that we start in state s and follow some policy, π.

This is a function that accepts a state, s, and returns the expected reward of starting in that state and taking actions according to our policy, π. It may not be immediately obvious why the value function depends on the policy. Consider that in our contextual bandit problem, if our policy was to choose entirely random actions (i.e., sample actions from a uniform distribution), the value (expected reward) of a state would probably be pretty low, since